

Luke Oleksiak

CS4420

Fall 2026

Assignment 5

The program, `assignment5.cc` is compiled by running “`g++ assignment5.cc`”. To use the program, use “`./a.out inputfilename [FCFS|SJF|RR] [timequantum]`”. FCFS for First come first serve, SJF for shortest job first, and RR for round robin scheduling algorithms respectively. Timequantum is only used with the round-robin algorithm, and trying to run the round robin algorithm without a positive integer time quantum (or no time quantum) will print an error message and exit the program. Likewise, not specifying a scheduling algorithm or an input file name (or an invalid input file name) will also print an error message and end the program. `input.1`, `input.3`, and `input.4` are all valid input files for the program, while `input.2` is an invalid input file used to check that the program will stop executing upon reading a file with an invalid number of processes specified in the file. `input.1` uses the input data of the sample input from the assignment.

Example compilation and program running with `input.1` with each scheduling algorithm:

(normal test case 1)

```
dinoguts@Lclasswork25:~/cs4420/assignment5$ ls
a.out  assignment5.cc  input.1  input.2
dinoguts@Lclasswork25:~/cs4420/assignment5$ g++ assignment5.cc
dinoguts@Lclasswork25:~/cs4420/assignment5$ ./a.out input.1 FCFS
```

PID	Arrival Time	Start Time	End Time	Running Time	Waiting Time
0	0	0	12	12	0
1	2	12	16	4	10
2	3	16	17	1	13
3	4	17	19	2	13

Average waiting Time: 9

```
dinoguts@Lclasswork25:~/cs4420/assignment5$ █
```

```
dinoguts@Lclasswork25:~/cs4420/assignment5$ ./a.out input.1 SJF
```

PID	Arrival Time	Start Time	End Time	Running Time	Waiting Time
0	0	0	12	12	0
2	3	12	13	1	9
3	4	13	15	2	9
1	2	15	19	4	13

Average Waiting Time: 7.75

```
dinoguts@Lclasswork25:~/cs4420/assignment5$
```

```
dinoguts@Lclasswork25:~/cs4420/assignment5$ ./a.out input.1 RR 5
```

PID	Start Time	End Time	Running Time
0	0	5	5
1	5	9	4
2	9	10	1
3	10	12	2
0	12	17	5
0	17	19	2

PID	Arrival Time	Running Time	End Time	Waiting Time
0	0	12	19	7
1	2	4	9	3
2	3	1	10	6
3	4	2	12	6

Average Waiting Time: 5.5

```
dinoguts@Lclasswork25:~/cs4420/assignment5$
```

Input.3 data:

```
3
0 1 5
1 2 3
2 10 4
```

Each algorithm running with input.3 data:

(normal test case 2)

```
dinoguts@Lclasswork25:~/cs4420/assignment5$ ./a.out input.3 FCFS
```

PID	Arrival Time	Start Time	End Time	Running Time	Waiting Time
Idle	-	0	1	1	
0	1	1	6	5	0
1	2	6	9	3	4
Idle	-	9	10	1	
2	10	10	14	4	0

Average waiting Time: 1.33333

```
dinoguts@Lclasswork25:~/cs4420/assignment5$ ./a.out input.3 SJF
```

PID	Arrival Time	Start Time	End Time	Running Time	Waiting Time
Idle	-	0	1	1	
0	1	1	6	5	0
1	2	6	9	3	4
Idle	-	9	10	1	
2	10	10	14	4	0

Average Waiting Time: 1.33333

```
dinoguts@Lclasswork25:~/cs4420/assignment5$ ./a.out input.3 RR 5
```

PID	Start Time	End Time	Running Time
Idle	0	1	1
0	1	6	5
1	6	9	3
Idle	9	10	1
2	10	14	4

PID	Arrival Time	Running Time	End Time	Waiting Time
0	1	5	6	0
1	2	3	9	4
2	10	4	14	0

Average Waiting Time: 1.33333

```
dinoguts@Lclasswork25:~/cs4420/assignment5$ █
```

Input.4 data

```
2
0 0 4
1 2 3
```

Each algorithm running with input.4 data:

(normal test case 3)

```
dinoguts@Lclasswork25:~/cs4420/assignment5$ ./a.out input.4 FCFS
```

PID	Arrival Time	Start Time	End Time	Running Time	Waiting Time
0	0	0	4	4	0
1	2	4	7	3	2

Average waiting Time: 1

```
dinoguts@Lclasswork25:~/cs4420/assignment5$ ./a.out input.4 SJF
```

PID	Arrival Time	Start Time	End Time	Running Time	Waiting Time
0	0	0	4	4	0
1	2	4	7	3	2

Average Waiting Time: 1

```
dinoguts@Lclasswork25:~/cs4420/assignment5$ ./a.out input.4 RR 2
```

PID	Start Time	End Time	Running Time
0	0	2	2
1	2	4	2
0	4	6	2
1	6	7	1

PID	Arrival Time	Running Time	End Time	Waiting Time
0	0	4	6	2
1	2	3	7	2

Average Waiting Time: 2

```
dinoguts@Lclasswork25:~/cs4420/assignment5$ █
```

Error handling test 1, invalid time quantum:

```
dinoguts@Lclasswork25:~/cs4420/assignment5$ ./a.out input.1 RR -4
```

Invalid time quantum

```
dinoguts@Lclasswork25:~/cs4420/assignment5$ █
```

Error handling test 2, missing time quantum:

```
dinoguts@Lclasswork25:~/cs4420/assignment5$ ./a.out input.1 RR
```

Error. Time quantum not found

```
dinoguts@Lclasswork25:~/cs4420/assignment5$ █
```

Error handling test 3, invalid number of arguments:

```
dinoguts@Lclasswork25:~/cs4420/assignment5$ ./a.out input.1
Invalid number of arguments
dinoguts@Lclasswork25:~/cs4420/assignment5$
```

Error handling test 4, invalid file name:

```
dinoguts@Lclasswork25:~/cs4420/assignment5$ ./a.out input.1 SJF
Could not open file
dinoguts@Lclasswork25:~/cs4420/assignment5$
```

Error handling test 5, invalid number of processes specified in file:

```
dinoguts@Lclasswork25:~/cs4420/assignment5$ ./a.out input.2 SJF
Invalid number of processes
dinoguts@Lclasswork25:~/cs4420/assignment5$
```

Tests of comparison functions logic:

Assertations and what cases they're testing:

```
assert(compareArrivals({0,0,5},{1,1,5}) == true);
assert(compareArrivals({0,0,5},{1,0,5}) == true);
assert(compareArrivals({1,1,5},{0,0,5}) == false);
assert(comparePID({0,0,5},{1,0,5}) == true);
assert(comparePID({1,0,5},{0,0,5}) == false);
assert(compareTimeRemaining({0,0,4},{1,1,5}) == true);
assert(compareTimeRemaining({0,0,8},{1,1,2}) == false);
assert(compareTimeRemaining({0,0,5},{1,1,5}) == true);
```

First assertion tests that sooner arrival time for left process for compareArrivals function results in function returning true.

Second assertion tests that same arrival time with lower PID for left process results for compareArrivals function results in function returning true.

Third assertion tests that sooner arrival time for right process for compareArrivals function results in function returning false.

Fourth assertion tests that smaller PID for left process for comparePID function results in function returning true.

Fifth assertion tests that smaller PID for right process for comparePID function results in function returning false.

Sixth assertion tests that lower time remaining for left process in compareTimeRemaining function results in function returning true.

Seventh assertion tests that lower time remaining for right process in compareTimeRemaining function results in function returning false.

Eighth assertion tests that same time remaining for both processes with a sooner arrival time for the left process in compareTimeRemaining function results in the function returning true.

Functions passing all assertions (program would throw an error during execution if any failed):

```
dinoguts@Lclasswork25:~/cs4420/assignment5$ g++ assignment5.cc
dinoguts@Lclasswork25:~/cs4420/assignment5$ ./a.out input.1 FCFS
```

PID	Arrival Time	Start Time	End Time	Running Time	Waiting Time
0	0	0	12	12	0
1	2	12	16	4	10
2	3	16	17	1	13
3	4	17	19	2	13

```
Average waiting Time: 9

Comparison function tests all passed as expected
dinoguts@Lclasswork25:~/cs4420/assignment5$ █
```